



National University of Computer and Emerging Sciences

Laboratory Manuals

for

Computer Networks - Lab

(CL-3001)

Department of Computer Science

FAST-NU, Lahore, Pakistan

Instructor:

Muhammad Nouman Hanif

Lab Manual 06

Objective:

Students should know:

- What a computer network is and what its advantages are.
- What is OSI Model?
- What is a socket?
- Client-Server Model
- TCP Socket Programming.

Introduction to TCP Socket Programming

Socket programming is a method for connecting two nodes on a network to communicate with each other. Sockets act as the endpoints of a bidirectional communications channel, providing a form of Inter-Process Communication (IPC). In the common client-server model, one node acts as a server, forming a "listener" socket on a specific port and IP address, while the client node reaches out to the server to establish a connection. This lab focuses on TCP (Transmission Control Protocol), which allows for reliable, connection-oriented communication between applications.

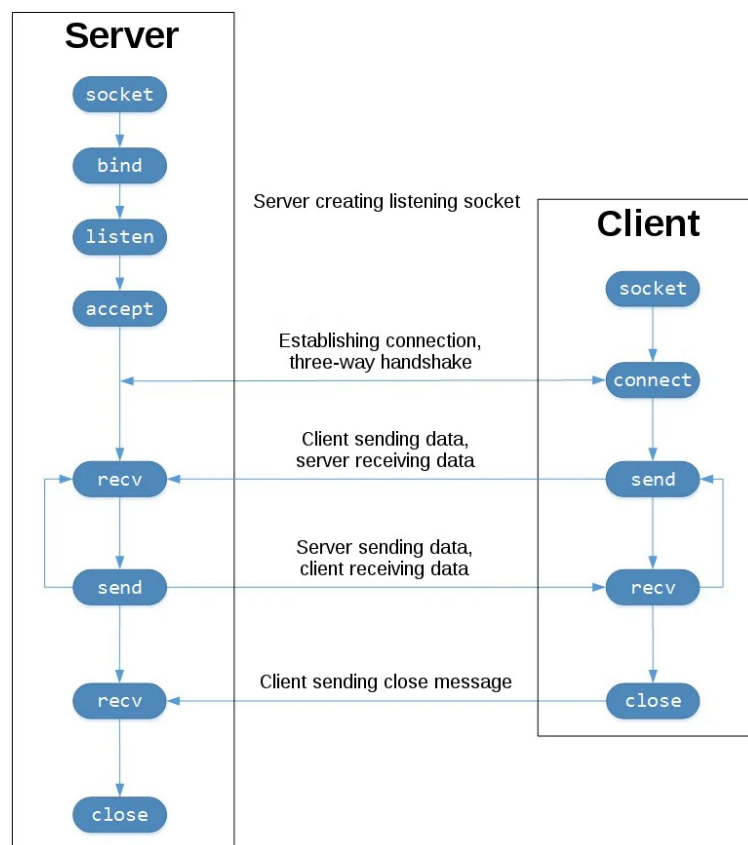


Figure 1: TCP Client-Server Communication Flow.

Key Differences in Practice: TCP vs. UDP Chat

Building a chat app with TCP is like having a **phone call**, while using UDP is like sending a series of **postcards**. Here's a breakdown of the main differences:

Feature	TCP (The Phone Call)	UDP (The Postcards)
Connection	A formal connection is established before any chat messages are sent. You know the other person is ready.	No connection is made. You just send messages and hope someone is there to receive them.
Reliability	Guaranteed delivery. If a message is lost, TCP automatically re-sends it. Your conversation will always be complete.	No guarantees. Messages can get lost and will never arrive. The chat would have missing sentences.
Ordering	Guaranteed order. Messages arrive in the exact sequence they were sent.	No guaranteed order. Messages can arrive jumbled (e.g., the reply before the question).
Simplicity	Easier for the developer. The protocol handles all the hard work of ensuring the chat is reliable and ordered.	Harder for the developer. You must write extra code to handle lost or out-of-order messages.

Bottom Line: For an application like text-based chat, where it's crucial that every message arrives correctly and in the right order, **TCP is the clear choice**.

The Connection Process

Establishing a TCP connection involves a sequence of steps and function calls on both the server and client sides.

Server-Side Setup: The server must first prepare to accept incoming connections. This involves a series of API calls:

1. `socket()`: Creates a new socket endpoint.
2. `bind()`: Associates the socket with a specific server address (IP and port number).
3. `listen()`: Puts the socket in listening mode, waiting for clients to connect.
4. `accept()`: Blocks execution and waits for a client. When a client connects, this call accepts the connection and returns a new socket object for communicating with that specific client.

Client-Side Connection: The client initiates the connection to the server:

1. `socket()`: Creates a new socket endpoint.
2. `connect()`: Attempts to establish a connection to the server's address and port. This call initiates the TCP three-way handshake.

TCP Three-Way Handshake: The `connect()` call triggers a three-step process to ensure both client and server are ready to communicate:

1. **SYN:** The client sends a SYN (synchronize) packet to the server to request a connection.
2. **SYN-ACK:** The server responds with a SYN-ACK (synchronize-acknowledge) packet, acknowledging the request.
3. **ACK:** The client sends a final ACK (acknowledge) packet, confirming the connection is established.

Once the handshake is complete, data can be exchanged using `send()` and `recv()` calls on both the client and server sockets.

Common Socket Methods

Python's `socket` module provides several functions to create client or server programs.

Server Socket Methods

Method	Description
<code>s.bind()</code>	Binds the socket to an address (hostname, port number pair).
<code>s.listen()</code>	Sets up and starts the TCP listener.
<code>s.accept()</code>	Passively accepts a TCP client connection, blocking until one arrives.

Client Socket Methods

Method	Description
<code>s.connect()</code>	Actively initiates a TCP server connection.

General Socket Methods

Method	Description
<code>s.recv()</code>	Receives a TCP message.
<code>s.send()</code>	Transmits a TCP message.
<code>s.close()</code>	Closes the socket connection.
<code>socket.gethostname()</code>	Returns the local hostname.

TCP Socket Programming Example

Here is a basic example of a TCP client and server written in Python 3.

Example 1: Simple TCP Chat Application

This example demonstrates a two-way chat. The server waits for one client to connect. Once connected, the server and client can send messages back and forth in the terminal until one of them types 'quit'.

Chat Server (chat_server.py)

```
1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 65432
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7     s.bind((HOST, PORT))
8     s.listen()
9     print(f"Chat server started, waiting for connection on port {PORT}...")
10    conn, addr = s.accept()
11    with conn:
12        print(f"Connected by {addr}")
13        while True:
14            data = conn.recv(1024)
15            if not data or data.decode().lower() == 'quit':
16                print("Client has left the chat.")
17                break
18            print(f"Client: {data.decode()}")
19
20            message = input("Server: ")
21            conn.sendall(message.encode())
22            if message.lower() == 'quit':
23                break
```

Chat Client (chat_client.py)

```
1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 65432
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7     s.connect((HOST, PORT))
8     print("Connected to chat server. Type 'quit' to exit.")
9     while True:
10        message = input("Client: ")
11        s.sendall(message.encode())
12
13        data = s.recv(1024)
14        if not data or message.lower() == 'quit':
15            print("Connection closed.")
16            break
17        print(f"Server: {data.decode()}")
```

Example 2: Simple TCP File Transfer

This example shows how to send a small file from a server to a client. The client requests a specific filename, and the server sends its contents. For this to work, create a file named `sample.txt` in the same directory as the server script.

File Server (file_server.py)

```
1 import socket
2
3 # Create a file named 'sample.txt' with some text in it.
4 HOST = '127.0.0.1'
5 PORT = 65433
6
7 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
8     s.bind((HOST, PORT))
9     s.listen()
10    print(f"File server listening on {PORT}...")
11    conn, addr = s.accept()
12    with conn:
13        print(f"Connected by {addr}")
14        # Receive the filename from the client
15        filename = conn.recv(1024).decode()
16        print(f"Client requested file: {filename}")
17
18        try:
19            with open(filename, 'rb') as f:
20                data = f.read()
21                conn.sendall(data)
22            print("File sent successfully.")
23        except FileNotFoundError:
24            error_message = "Error: File not found."
25            conn.sendall(error_message.encode())
26            print(error_message)
```

File Client (file_client.py)

```
1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 65433
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
7     s.connect((HOST, PORT))
8
9     # Request a file from the server
10    filename_to_request = "sample.txt"
11    s.sendall(filename_to_request.encode())
12
13    # Receive the file data
14    data = s.recv(4096)
15
16    # Save the received data to a new file
17    with open("received_file.txt", "wb") as f:
18        f.write(data)
19
20    print(f"File '{filename_to_request}' received and saved as 'received_file.txt'.")
```

TCP Echo Server

This script creates a TCP/IP socket, binds it to 'localhost' on port 10000, and listens for a connection. When a client connects, it receives data (bytes) and sends the exact same data back.

```
1 import socket
2 import struct # Used to convert numbers to bytes and back
3
4 def recv_all(sock, n):
5     """Helper function to receive n bytes or return None if EOF is hit"""
6     data = bytearray()
7     while len(data) < n:
8         packet = sock.recv(n - len(data))
9         if not packet:
10             return None
11         data.extend(packet)
12     return data
13
14 # Create a TCP/IP socket
15 sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
16
17 # Bind the socket to the port
18 server_address = ('localhost', 10000)
19 print(f'Starting up on {server_address[0]} port {server_address[1]}')
20 sock.bind(server_address)
21 sock.listen(1)
22
23 while True:
24     print('Waiting for a connection')
25     connection, client_address = sock.accept()
26     try:
27         print(f'Connection from {client_address}')
28
29         # 1. Receive the 8-byte header first
30         msg_len_header = recv_all(connection, 8)
31         if not msg_len_header:
32             break
33
34         # 2. Unpack the header to get the message length
35         # !Q means: Network Byte Order, Unsigned Long Long (8 bytes)
36         msg_len = struct.unpack('!Q', msg_len_header)[0]
37
38         # 3. Receive the full message based on the length from the header
39         full_message = recv_all(connection, msg_len)
40
41         print(f'Received full message: {full_message!r}')
42
43         # Echo the full message back (including its header)
44         print('Sending data back to the client')
45         connection.sendall(msg_len_header + full_message)
46
47     finally:
48         print("Cleaning up connection")
49         connection.close()
```

Listing 1: Tcp_Echo_Server.py

TCP Echo Client

The client connects to the server, sends a message as a byte string, receives the echo back, and then closes the connection.

```
1 import socket
2 import struct # Used to convert numbers to bytes and back
3 import sys
4
5 # Create a TCP/IP socket
6 sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7
8 # Connect the socket to the port where the server is listening
9 server_address = ('localhost', 10000)
10 print(f'Connecting to {server_address[0]} port {server_address[1]}')
11 sock.connect(server_address)
12
13 try:
14     message = b'This is the message. It will be repeated.'
15
16     # 1. Get the length of the message
17     msg_len = len(message)
18
19     # 2. Pack the length into an 8-byte header
20     # !Q means: Network Byte Order, Unsigned Long Long (8 bytes)
21     header = struct.pack('!Q', msg_len)
22
23     # 3. Send the header first, then the message
24     print(f'Sending message: {message!r}')
25     sock.sendall(header + message)
26
27     # Look for the response (no changes needed here from your original)
28     amount_received = 0
29     amount_expected = msg_len + 8 # We expect the header back, too
30
31     full_response = b''
32     while amount_received < amount_expected:
33         data = sock.recv(16)
34         amount_received += len(data)
35         full_response += data
36
37     # We can now process the full response
38     print(f'Received full response: {full_response!r}')
39
40 finally:
41     print('Closing socket')
42     sock.close()
```

Listing 2: Tcp_Echo_Client.py

The client sends a '[header][message]' package, and the server receives it and echoes the exact same '[header][message]' package back.

Lab Exercises

Task 1: Simple Client-Server ID Exchange

Write a TCP client and server that can communicate predefined messages to each other, including a client ID.

Requirements

- The client should send a message in the format: "Hello I am client and My id is X", where X is a single digit from 0 to 9.
- The server should respond with: "Hello I am server. Your received id is X".
- The client and server should run on the same machine.
- The server must run indefinitely, handling multiple clients one by one, without terminating after serving a single client.

Sample Test Bench

Client1 sends: "Hello I am client and My id is 1"

Server response on client1: "Hello I am server. Your received id is 1"

Client2 sends: "Hello I am client and My id is 2"

Server response on client2: "Hello I am server. Your received id is 2"

Task 2: Vowel-Based String Inversion

Write a TCP client-server program where the client sends a string, and both the server and client perform conditional, word-by-word inversion on it.

Requirements

- The client sends a string to the server (e.g., "the birds fly in dry sky at night").
- The server inverts only the words in the string that contain one or more vowels (e.g., "the" becomes "eht", "birds" becomes "sdirb").
- The server sends this modified string back to the client.
- The client receives the string from the server and then inverts only the words that contain **no** vowels (e.g., "fly" becomes "ylf", "dry" becomes "yrd").
- The client then displays the final string on its terminal.
- The server must run indefinitely and handle clients sequentially.

Sample Test Bench

Client sends to server: "the birds fly in dry sky at night"

Server displays and returns to client:

"eht sdirb fly ni dry sky ta thgin"

Client displays the final string:
"eht sdirb ylf ni yrd yks ta thgin"

Task 3: Remote File Word Counter

Create a client-server application where the client can ask the server to count the words in a file stored on the server's machine.

Requirements

- The server should have access to a few text files (e.g., `story1.txt`, `poem.txt`).
- The client prompts the user to enter a filename to analyze.
- The client sends the requested filename to the server.
- The server receives the filename, opens and reads the file, counts the total number of words, and sends the count back to the client as a string (e.g., "The file contains 127 words.").
- If the file does not exist on the server, the server should send back an appropriate error message.
- The client receives the server's response and prints it to the console.

Sample Test Bench

Assume `story1.txt` on the server contains "The quick brown fox jumps over the lazy dog."

-- Client --

Enter filename to count words in: `story1.txt`

-- Server --

Received request for file: `story1.txt`

File contains 9 words. Sending result.

-- Client Output --

The server reports: The file contains 9 words.

Task 4: Number Guessing Game

Implement a stateful number guessing game that demonstrates a persistent TCP connection.

Requirements

- When a client connects, the server must generate a random integer between 1 and 100. This number must be unique for each client connection.
- The client will then repeatedly prompt the user to enter a guess and will send that guess to the server.
- The server compares the guess to its secret number and responds with one of three strings: "Higher", "Lower", or "Correct!".

- The client prints the server's response after each guess.
- The game continues over the same TCP connection until the client guesses the number correctly.
- When the correct number is guessed, the server sends a final congratulatory message (e.g., "Correct! You guessed the number in 8 tries.") and then closes the connection.

Sample Test Bench

Assume the server generates the secret number 42.

```
-- Client --
```

```
Connected to the guessing game server.
```

```
Enter your guess (1-100): 50
```

```
-- Server Response to Client --
```

```
Lower
```

```
-- Client --
```

```
Enter your guess (1-100): 25
```

```
-- Server Response to Client --
```

```
Higher
```

```
-- Client --
```

```
Enter your guess (1-100): 42
```

```
-- Server Response to Client --
```

```
Correct! You guessed the number in 3 tries.
```

```
(Server closes connection)
```